v1.0.0
20260324

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[Lab Notes]
JAVA VIRTUAL THREADS

REFERENCES

- Slides based on *Virtual Threads: New Foundations for High-Scale Java Applications*, by B. Goetz and D. Bryant
  - https://www.infoq.com/articles/java-virtual-threads/

VIRTUAL THREADS IN A SLIDE

- Virtual threads are a *lightweight implementation of Java threads*
  - main deliverable of OpenJDKs **Project Loom**
  - **JEP 425: Virtual Threads -** https://openjdk.org/jeps/425
    - JEP = JDK Enhancement Proposals (https://openjdk.org/jeps/0)
- Key point
  - threads as **logical unit of concurrency**
  - …but preserving the benefits of classic 'physical' threads

PLATFORM THREADS

- Most JVM implementations today implement Java threads as thin wrappers around OS-managed threads
  - these are called *platform threads*
- Good points
  - reusing all the power of the thread abstraction and mechanisms directly offered by the OS
- Bad points
  - thread creation expensive and resource-heavy

    => practical limit on *how many* we can create

    => on how we use threads in our programs

WHY RESOURCE-HEAVY? THE STACK IN PLATFORM THREADS

- In OS the thread **stack** is allocated as a monolithic block of memory at thread creation time that cannot be resized later

  => each thread => ~megabyte-scale chunks of memory to manage the native and Java call stacks
- Stack size can be tuned both with command-line switches and Thread constructors, but tuning is risky in both directions
  - if stacks are *overprovisioned*, we will use even more memory;
  - if they are *underprovisioned*, we risk StackOverflowException if the wrong code is called at the wrong time
- We generally lean towards overprovisioning thread stacks as being the lesser of evil, *but the result is a relatively low limit on how many concurrent threads we can have for a given amount of memory*

IMPACT ON PROGRAMMING

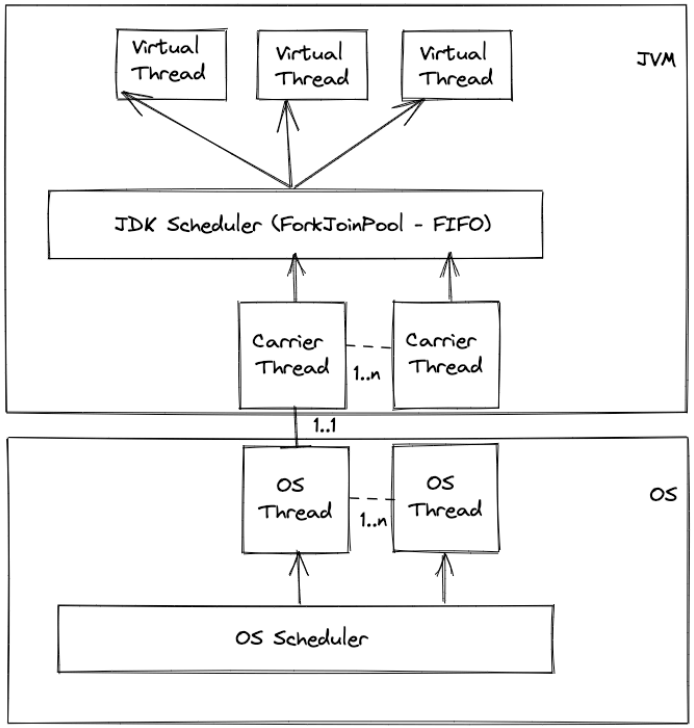
- Limiting how many threads complicates the design and development of applications
  - e.g in server side application, it would be natural to adopt a thread-per-task approach, i.e. assigning each incoming request to a single thread for the lifetime of the task
- *Aligning the applications unit of concurrency (the task) with the platforms (the thread)* would maximize ease of development, debugging, and maintenance
- Unfortunately, as programs scale, this approach is on a collision course with the memory characteristics of platform threads.
  - Thread-per-task scales well enough for moderate-scale applications -- we can easily service 1000 concurrent requests…
  - …but we will not be able to service 1M concurrent requests using the same technique, even if the hardware has adequate CPU capacity and IO bandwidth
- Virtual threads have been designed to this purpose

VIRTUAL THREADS

- Technically, they are an alternative implementation of java.lang.Thread, in which:
  - stack frames are in Javas garbage-collected heap
  - the memory footprint starts out at only a few hundred bytes and can be expanded and shrunk automatically
- Virtual threads are so-named because they share characteristics with *virtual memory*
  - with virtual memory, applications have the illusion to have access to the entire memory address space, beyond the available physical memory
    - the hardware completes this illusion by temporarily mapping plentiful virtual memory to scarce physical memory as needed, and when some other virtual page needs that physical memory, the old contents are first paged out to disk
  - similarly, virtual threads are cheap and plentiful, and share the scarce and expensive platform threads as needed, and inactive virtual thread stacks are "paged" out to the heap.
- Virtual threads are similar to user-mode threads found in other languages (e.g. goroutines in Go, processes in Erlang), but have the advantage of being semantically identical to the threads we already have

HOW VIRTUAL THREADS ARE MANAGED: CARRIER THREADS

- The operating system only knows about platform threads, which remain the unit of scheduling
- To run code in a virtual thread, the Java runtime arranges for it to run by *mounting it on some platform thread, called a* ***carrier thread***



MOUNTING AND UNMOUNTING

- ***Mounting*** a virtual thread
  - the needed stack frames from the heap is copied to the stack of the carrier thread, borrowing the carrier stack while it is mounted.
- When code running in a virtual thread would otherwise block for IO, locking, or other resource availability, it can be ***unmounted*** from the carrier thread
  - any modified stack frames are copied back to the heap, freeing the carrier thread for something else
    - …such as running another virtual thread
  - *nearly all blocking points in the JDK have been adapted so that when encountering a blocking operation on a virtual thread, the virtual thread is unmounted from its carrier instead of blocking.*

MOUNTING AND UNMOUNTING

- Mounting and unmounting a virtual thread on a carrier thread is an implementation detail that is entirely invisible to Java code
  - Java code cannot observe the identity of the current carrier (calling Thread::currentThread always returns the virtual thread)
  - the stack frames of the carrier do not show up in exceptions or thread dumps for the virtual thread
- During the virtual threads lifetime, it may run on many different carrier threads
  - but anything depending on thread identity, such as locking, will see a consistent picture of what thread it is running on

THE API LEVEL

- Virtual threads have relatively little new API surface
- There are several new methods for creating virtual threads
  - **Thread::ofVirtual**
    - started, unstated method
  - **Executors.newVirtualThreadPerTaskExecutor**
- However after creation, *they are ordinary Thread objects and behave like the threads we already know*
  - Existing APIs such as Thread::currentThread, ThreadLocal, interruption, stack walking, etc, work exactly the same on virtual threads as on platform threads, which means we can run existing code confidently on virtual threads

SCALABILITY

- Virtual Threads are especially good to scale in applications doing I/O e.g. sever application
- Despite the difference in creation costs, virtual threads *are not faster than platform threads* (obviously)
  - …so we can’t do any more computation with one virtual thread in one second than we can with a platform thread
  - …nor can we schedule any more actively running virtual threads than we can platform threads
    - both are limited by the number of available CPU cores
- **However, because they are so lightweight, we can have many more inactive virtual threads than we can with platform threads**
  - requests in server applications spend much more time doing network, file, or database I/O than computation
    - so if we run each task in its own thread, most of the time that thread will be blocked on I/O or other resource availability

  => virtual threads allow IO-bound thread-per-task applications to scale better by removing the most common scaling bottleneck — the maximum number of threads -- which in turn enables better hardware utilization

LITTLE LAW

- The scalability of a stable system is governed by **Littles Law**, which relates *latency*, *concurrency*, and *throughput*
  - throughput **T** = number of tasks executed per second
  - If each request has a duration (or latency) of **d**, and we can perform **N** tasks concurrently, then throughput **T** is given by

$$T = N / d$$

- Remarks
  - Littles Law doesn't care about what portion of the time is spent "doing work" vs "waiting", or whether the unit of concurrency is a thread, a CPU, an ATM machine, or a human bank teller….
  - It just states that ***to scale up the throughput, we either have to proportionally scale down the latency or scale up the number of requests we can handle concurrently.***
- **When we hit the limit on concurrent threads, the throughput of the thread-per-task model is limited by Littles Law.**
- Virtual threads address this in a graceful way by giving us more concurrent threads rather than asking us to change our programming model

EXAMPLE

- The following example creates 100,000 virtual threads that simulate an IO-bound operation by sleeping for one second
- It creates a virtual-thread-per-task executor and submits the tasks as lambdas.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 100_000).forEach(i -> {
        executor.submit(() -> {
            Thread.sleep(Duration.ofSeconds(1));
            return i;
        });
    });
}  // close() called implicitly
```

- On normal desktop system with no special configuration options, running this program takes about 1.6 seconds in a cold start, and about 1.1 seconds after warmup. If we try running this program with a cached thread pool instead, depending on how much memory is available, it may well crash with OutOfMemoryError before all the tasks are submitted. And if we ran it with a fixed-sized thread pool with 1000 threads, it wont crash, but Littles Law accurately predicts it will take 100 seconds to complete

BLOCKING AND UNMOUNTING: CURRENT LIMITATIONS (1/2)

- Typically, a virtual thread will unmount when
  - it blocks on I/O
    - e.g. when it reads from a socket
  - when it calls other blocking operations in the JDK
    - e.g. take on a BlockingQueue
  - In these cases, when the blocking operation is ready to finish (e.g. the socket received the bytes or the queue can hand out an element), the operation submits the virtual thread back to the scheduler, which will, in FIFO order, eventually mount it to resume execution
- However **not all blocking operations in the JDK unmount the virtual thread.**
  - some capture the carrier thread and the underlying OS platform thread, thus blocking both
  - the capture of an OS thread is compensated by temporarily adding a platform thread to the scheduler, which can hence occasionally exceed the number of available processors
    - a maximum can be specified with a system property

CURRENT LIMITATIONS: AN IMPORTANT CASE - PINNED THREADS

- Besides the previous cases, there is an important case which does not support unmounting:

> *when a virtual thread executes a native method or a foreign function or it executes code inside a* ***synchronized block*** *or method, the virtual thread will be* pinned *to its carrier thread*

- A **pinned thread** will not unmount in situations where it otherwise would
  - no platform thread is added to the scheduler in this situation, though
- Capturing operations and pinned threads will reintroduce platform threads that are waiting for something to finish

  => this doesn't make an application incorrect, but it might hinder its scalability.

IMPLEMENTING MONITORS FOR VIRTUAL THREADS

- The problem of thread pinning affects the way in which we implement monitors
  - if we implement monitors using synchronized blocks, then we block the platform thread, not the virtual one
- This issue is solved by implementing monitors using JDK classes that have been revised to support unmounting
  - in this case: **ReentrantLock** and **Condition**